

PROGRAM

```
import numpy as np

import matplotlib.pyplot as plt

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([
    [0],
    [1],
    [1],
    [0]
])

np.random.seed(42)

input_neurons = 2
hidden_neurons = 2
output_neurons = 1

W1 = np.random.randn(input_neurons, hidden_neurons)
B1 = np.zeros((1, hidden_neurons))

W2 = np.random.randn(hidden_neurons, output_neurons)
B2 = np.zeros((1, output_neurons))

learning_rate = 0.1

epochs = 10000

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

```

def sigmoid_derivative(x):
    return x * (1 - x)

loss_history = []

for epoch in range(epochs):
    hidden_input = np.dot(X, W1) + B1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + B2
    predicted_output = sigmoid(final_input)
    loss = -np.mean(
        y * np.log(predicted_output + 1e-8) +
        (1 - y) * np.log(1 - predicted_output + 1e-8)
    )
    loss_history.append(loss)
    output_error = predicted_output - y
    dW2 = np.dot(hidden_output.T, output_error)
    dB2 = np.sum(output_error, axis=0, keepdims=True)
    hidden_error = np.dot(output_error, W2.T) * sigmoid_derivative(hidden_output)
    dW1 = np.dot(X.T, hidden_error)
    dB1 = np.sum(hidden_error, axis=0, keepdims=True)
    W2 -= learning_rate * dW2
    B2 -= learning_rate * dB2
    W1 -= learning_rate * dW1
    B1 -= learning_rate * dB1
    if epoch % 1000 == 0:
        print(f'Epoch {epoch}, Loss = {loss:.6f}')
print("\nPredictions after Training")
for i in range(len(X)):

```

```

hidden = sigmoid(np.dot(X[i], W1) + B1)
output = sigmoid(np.dot(hidden, W2) + B2)
prediction = 1 if output >= 0.5 else 0
print(
    f"Input: {X[i]} "
    f"Predicted: {prediction} "
    f"Actual: {y[i][0]} "
)
plt.figure(figsize=(8, 5))
plt.plot(loss_history)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid(True)
plt.show()

x_min, x_max = -0.5, 1.5
y_min, y_max = -0.5, 1.5
xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.01),
    np.arange(y_min, y_max, 0.01)
)
grid_points = np.c_[xx.ravel(), yy.ravel()]
hidden = sigmoid(np.dot(grid_points, W1) + B1)
output = sigmoid(np.dot(hidden, W2) + B2)
Z = (output > 0.5).astype(int)
Z = Z.reshape(xx.shape)

```

```
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.4)
for i in range(len(X)):
    if y[i] == 1:
        plt.scatter(
            X[i, 0],
            X[i, 1],
            marker='o',
            s=200,
            label='Class 1' if i == 1 else ""
        )
    else:
        plt.scatter(
            X[i, 0],
            X[i, 1],
            marker='s',
            s=200,
            label='Class 0' if i == 0 else ""
        )
plt.title("XOR Decision Boundary Learned by DNN")
plt.xlabel("Input 1")
plt.ylabel("Input 2")
plt.legend()
plt.grid(True)
plt.show()
```

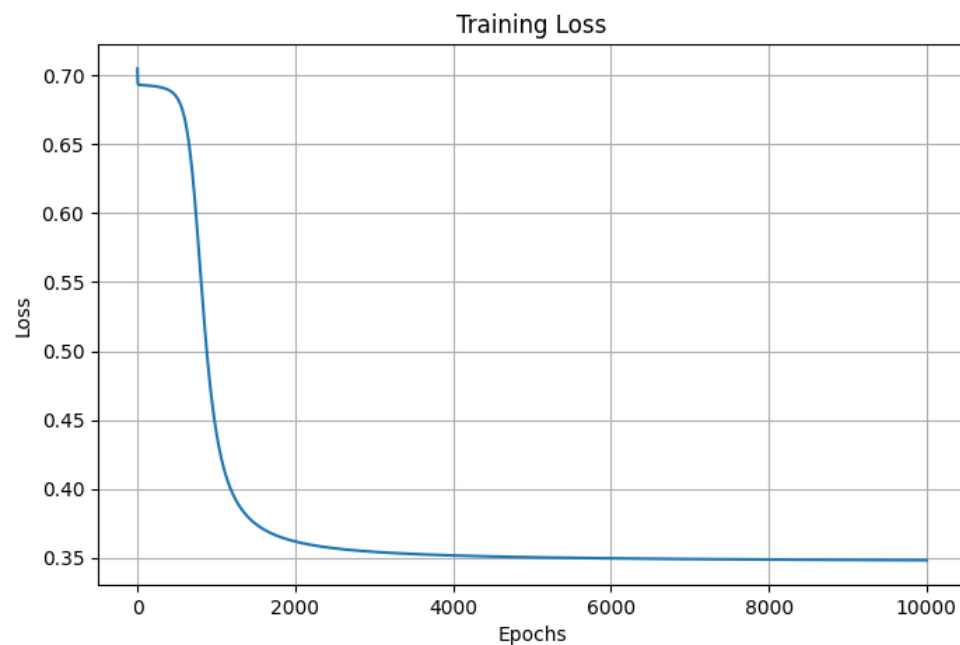
OUTPUT

```
Epoch 0, Loss = 0.704922  
Epoch 1000, Loss = 0.440706  
Epoch 2000, Loss = 0.361744  
Epoch 3000, Loss = 0.354224  
Epoch 4000, Loss = 0.351610  
Epoch 5000, Loss = 0.350303  
Epoch 6000, Loss = 0.349525  
Epoch 7000, Loss = 0.349010  
Epoch 8000, Loss = 0.348645  
Epoch 9000, Loss = 0.348373
```

Predictions after Training

```
Input: [0 0] Predicted: 0 Actual: 0  
Input: [0 1] Predicted: 0 Actual: 1  
Input: [1 0] Predicted: 1 Actual: 1  
Input: [1 1] Predicted: 1 Actual: 0
```

Figure 1



(x, y) = (9.86e+03, 0.494)

